

Week 5

A – Class Discussions

Big Data

What are the characteristics of Big Data?

- Big Data is characterized by its volume, velocity, variety, veracity, value, and variability (often referred to as the Six V's).
- These traits describe the challenges and opportunities of processing and analyzing massive, complex datasets that traditional data management tools cannot handle efficiently.

What are the (3 then 4 then) Six V's of Big Data?

- Original 3 V's:
 1. Volume – The enormous scale of data.
 2. Velocity – The speed at which data is generated and processed.
 3. Variety – The diverse types of data (structured, unstructured, semi-structured).
- Expanded to 4 V's:
 4. Veracity – The uncertainty or trustworthiness of data.
- Now 6 V's:
 5. Value – The usefulness of data in decision-making.
 6. Variability – Inconsistencies in data flow or meaning.

Who/What is generating these large Vs of data?

- **Humans:** Social media, online transactions, emails.
- **Machines/IoT:** Sensors, smart devices, industrial equipment.
- **Organizations:** Business transactions, healthcare records, scientific research.
- **Automated Systems:** AI, logs, surveillance cameras, satellites.

Big Data and Cloud Computing.

How are IoT, Cloud computing and Big Data linked?

- **IoT (Internet of Things)** generates massive amounts of data from sensors and devices.
- **Cloud Computing** provides the storage, processing power, and scalability needed to handle this data.
- **Big Data** technologies analyze and extract insights from the collected IoT data.
 - **IoT** → Produces data (Volume, Velocity, Variety).
 - **Cloud** → Stores & processes data (Scalability, On-demand resources).
 - **Big Data** → Analyzes data (Machine Learning, Real-time analytics).

Together, they enable smart applications like predictive maintenance, smart cities, and real-time monitoring.

What is NoSQL?

- NoSQL (Not Only SQL) is a type of database designed for unstructured, distributed, and large-scale data.
- Unlike traditional SQL databases (which use tables), NoSQL databases use flexible models like:
 - Key-Value (e.g., Redis)
 - Document (e.g., MongoDB)
 - Column-Family (e.g., Cassandra)
 - Graph (e.g., Neo4j)
- Handles Big Data efficiently.
- Scales horizontally (across servers).
- Supports high-speed read/write operations.

What are the characteristics and advantages of using NoSQL for scalable IoT applications?

- **Characteristics of NoSQL for IoT:**

- **Schema-less** → Adapts to varying IoT data formats (JSON, XML, sensor data).
- **Distributed & Scalable** → Handles massive IoT device data across clusters.
- **High Performance** → Low latency reads/writes for real-time analytics.
- **Flexible Data Models** → Stores structured, semi-structured, and unstructured data.
- **Advantages for IoT:**
 - **Handles High Velocity** – Billions of sensor readings per second.
 - **Scalability** – Easily expands with growing IoT networks.
 - **Fault Tolerance** – Replicates data across servers for reliability.
 - **Cost-Effective** – Uses commodity hardware (unlike expensive SQL setups).

Data Warehousing

What is Data Warehousing?

- **Data Warehousing** is the process of collecting, storing, and managing structured data from multiple sources in a centralized repository optimized for **analytics and reporting** (rather than transaction processing).
- **Key Characteristics:**
 - **Integrated** – Consolidates data from different sources (databases, IoT sensors, logs).
 - **Time-variant** – Tracks historical data for trend analysis.
 - **Non-volatile** – Data is read-only once stored (not frequently updated/deleted).
 - **Subject-oriented** – Organized by business domains (e.g., sales, IoT sensor data).

Why might we need Data Warehousing for building scalable IoT applications?

- **Challenges in IoT Data:**
 - **Massive Volume** – Billions of sensor readings daily.
 - **Multiple Sources** – Devices, APIs, legacy systems.
 - **Need for Historical Analysis** – Detecting patterns (e.g., predictive maintenance).

- **How Data Warehousing Helps IoT:**
 - **Centralized Storage** – Aggregates dispersed IoT data for unified analysis.
 - **Fast Query Performance** – Optimized for analytics (unlike raw databases).
 - **Historical Insights** – Stores long-term data for trend detection.
 - **Integration with BI Tools** – Powers dashboards (e.g., Tableau, Power BI).
 - **Scalability** – Cloud-based warehouses (BigQuery, Redshift) handle IoT's growing data.
- Use Case Example: A smart factory uses a data warehouse to:
 - Analyze sensor data for equipment failures.
 - Optimize energy usage over time.
 - Generate reports on operational efficiency.

B – Group Activities

Design a JSON string for each of the following:

1. A room temperature sensor for the room you are in.

```
{  
  "sensor_id": "temp_001",  
  "location": "living_room",  
  "timestamp": "2023-11-15T14:30:45Z",  
  "temperature_celsius": 22.5,  
  "humidity_percent": 45,  
  "battery_level": 85,  
  "status": "normal"  
}
```

2. The status of a self-driving car.

```
{
  "vehicle_id": "sd_car_xyz123",
  "timestamp": "2023-11-15T14:31:22Z",
  "location": {
    "latitude": 34.052235,
    "longitude": -118.243683
  },
  "speed_kmh": 45,
  "fuel_level": 78,
  "autonomous_mode": true,
  "system_status": "operational",
  "current_destination": "Downtown LA",
  "sensor_health": {
    "lidar": "ok",
    "camera": "ok",
    "radar": "ok"
  }
}
```

3. The location of a person.

```
{
  "person_id": "user_789",
  "timestamp": "2023-11-15T14:32:10Z",
  "location": {
    "latitude": 40.7128,
    "longitude": -74.0060,
    "accuracy_meters": 5
  },
}
```

4. A weather update for a location.

```
{
  "station_id": "weather_nyc_001",
  "timestamp": "2023-11-15T14:33:05Z",
  "location": {
```

```
"city": "New York",
"country": "USA",
"coordinates": {
  "latitude": 40.7128,
  "longitude": -74.0060
},
"temperature_c": 18,
"humidity_percent": 65,
"wind_speed_kmh": 15,
"conditions": "partly_cloudy",
"air_quality_index": 42,
"forecast": {
  "next_3_hours": "light_rain",
  "next_12_hours": "cloudy"
}
}
```

5. The status of a smart vacuum cleaner.

```
{
  "device_id": "vacuum_living_room_1",
  "timestamp": "2023-11-15T14:35:20Z",
  "status": "cleaning",
  "battery_percent": 73,
  "current_location": {
    "room": "living_room",
    "x_coord": 2.5,
    "y_coord": 3.1
  },
  "dust_bin_level": 45,
  "current_mode": "auto",
  "schedule": {
    "next_clean": "2023-11-15T20:00:00Z"
  },
}
```

```
"error_status": null
}
```

6. A TV program change request for a TV.

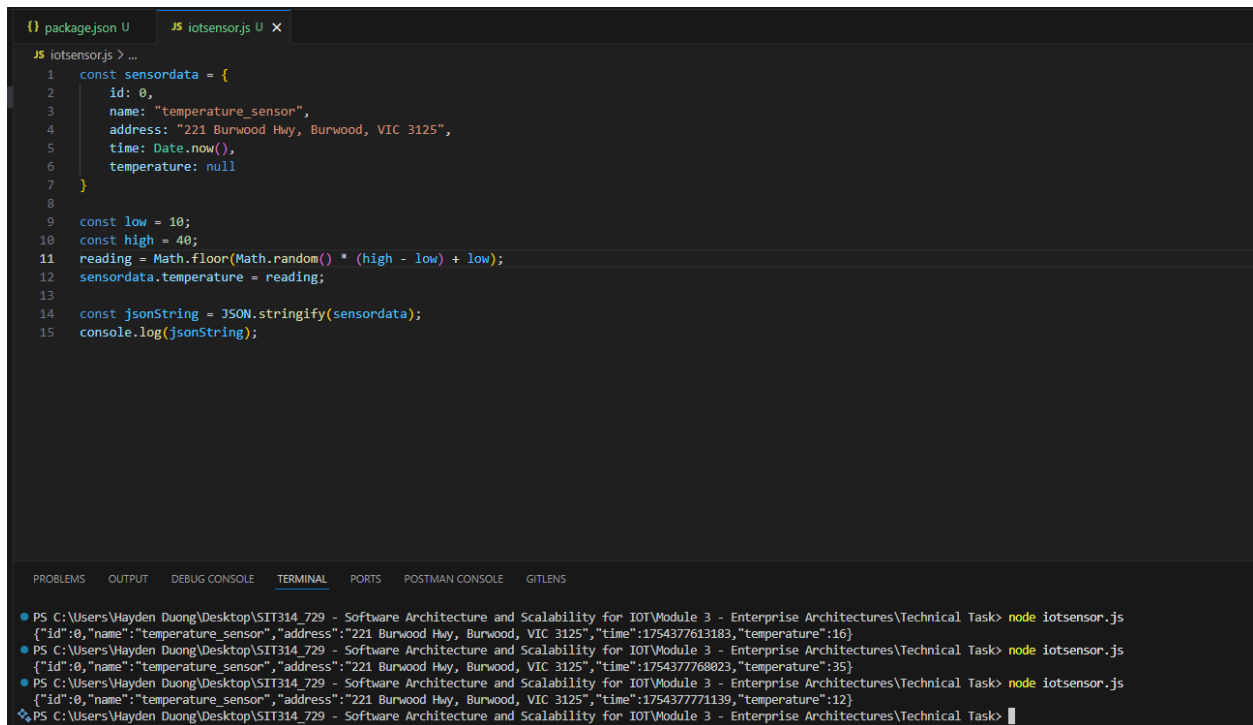
```
{
  "request_id": "tv_req_456",
  "timestamp": "2023-11-15T14:36:40Z",
  "device_id": "living_room_tv",
  "user_id": "family_member_3",
  "current_channel": "HBO",
  "requested_channel": "National Geographic",
}
```

7. The request message to set an air-conditioning setting.

```
{
  "request_id": "ac_set_789",
  "timestamp": "2023-11-15T14:37:15Z",
  "device_id": "bedroom_ac_unit",
  "requested_settings": {
    "power": "on",
    "mode": "cool",
    "temperature_c": 22,
    "fan_speed": "medium",
    "swing": "auto"
  },
  "scheduled_duration_minutes": 120,
  "energy_saving_mode": true,
  "request_source": "mobile_app"
}
```

C - Technical Tasks

Instructions: Creating Sensor Readings



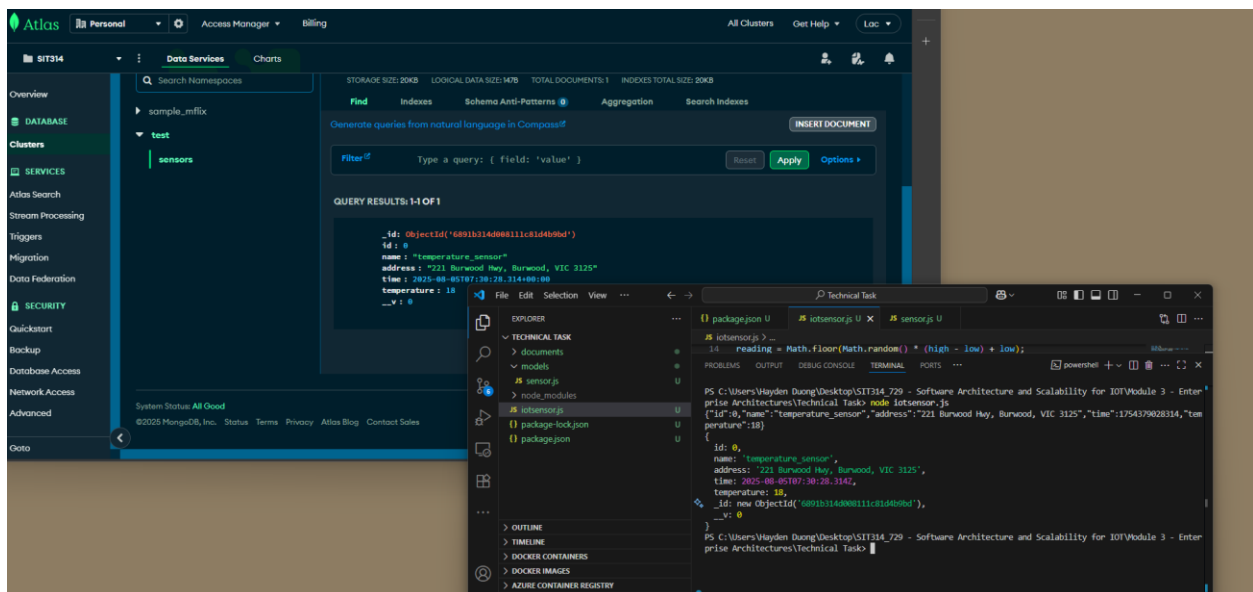
```
1 const sensordata = {
2   id: 0,
3   name: "temperature_sensor",
4   address: "221 Burwood Hwy, Burwood, VIC 3125",
5   time: Date.now(),
6   temperature: null
7 }
8
9 const low = 10;
10 const high = 40;
11 reading = Math.floor(Math.random() * (high - low) + low);
12 sensordata.temperature = reading;
13
14 const jsonString = JSON.stringify(sensordata);
15 console.log(jsonString);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE GITLENS

- PS C:\Users\Hayden Duong\Desktop\SIT314_729 - Software Architecture and Scalability for IOT\Module 3 - Enterprise Architectures\Technical Task> node iotsensor.js
- {"id":0,"name":"temperature_sensor","address":"221 Burwood Hwy, Burwood, VIC 3125","time":1754377613183,"temperature":16}
- PS C:\Users\Hayden Duong\Desktop\SIT314_729 - Software Architecture and Scalability for IOT\Module 3 - Enterprise Architectures\Technical Task> node iotsensor.js
- {"id":0,"name":"temperature_sensor","address":"221 Burwood Hwy, Burwood, VIC 3125","time":1754377768023,"temperature":35}
- PS C:\Users\Hayden Duong\Desktop\SIT314_729 - Software Architecture and Scalability for IOT\Module 3 - Enterprise Architectures\Technical Task> node iotsensor.js
- {"id":0,"name":"temperature_sensor","address":"221 Burwood Hwy, Burwood, VIC 3125","time":1754377771139,"temperature":12}

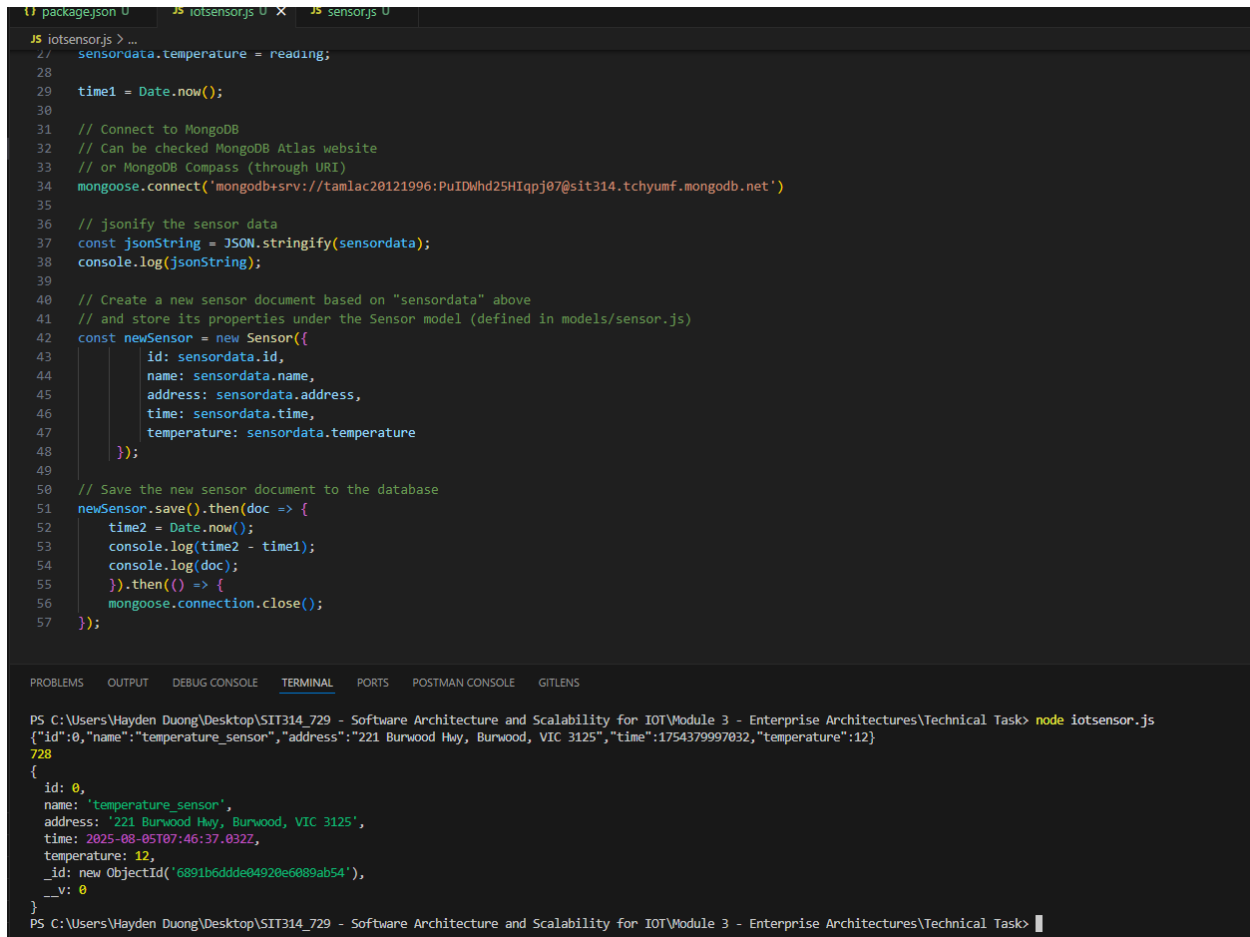
Picture 1 – Results gained from random temperature generator code.

Instructions: Storing Sensor Readings



Picture 2 – Successfully create a new Sensor and stored on MongoDB Atlas

Scalability of IoT Data Storage



```
JS iotsensor.js > ...
27 sensordata.temperature = reading;
28
29 time1 = Date.now();
30
31 // Connect to MongoDB
32 // Can be checked MongoDB Atlas website
33 // or MongoDB Compass (through URI)
34 mongoose.connect('mongodb+srv://tamlac20121996:PuIDWhd25HIqpj07@sit314.tchymf.mongodb.net')
35
36 // jsonify the sensor data
37 const jsonString = JSON.stringify(sensordata);
38 console.log(jsonString);
39
40 // Create a new sensor document based on "sensordata" above
41 // and store its properties under the Sensor model (defined in models/sensor.js)
42 const newSensor = new Sensor({
43   id: sensordata.id,
44   name: sensordata.name,
45   address: sensordata.address,
46   time: sensordata.time,
47   temperature: sensordata.temperature
48 });
49
50 // Save the new sensor document to the database
51 newSensor.save().then(doc => {
52   time2 = Date.now();
53   console.log(time2 - time1);
54   console.log(doc);
55 }).then(() => {
56   mongoose.connection.close();
57 });
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE GITLENS

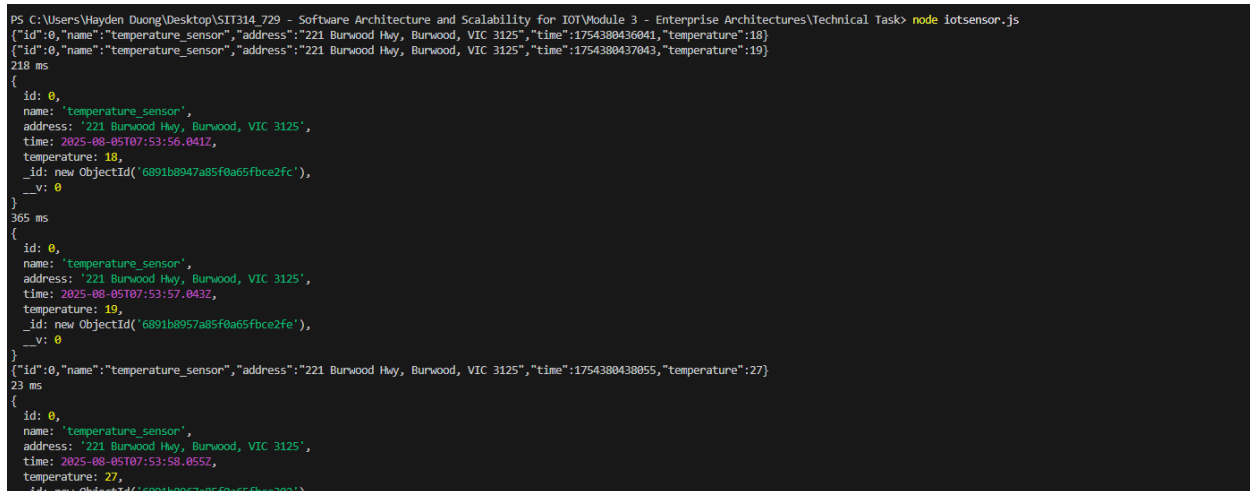
PS C:\Users\Hayden Duong\Desktop\SIT314_729 - Software Architecture and Scalability for IOT\Module 3 - Enterprise Architectures\Technical Task> node iotsensor.js

728

```
{
  id: 0,
  name: 'temperature_sensor',
  address: '221 Burwood Hwy, Burwood, VIC 3125',
  time: 2025-08-05T07:46:37.032Z,
  temperature: 12,
  _id: new ObjectId('6891b6ddde04920e6089ab54'),
  __v: 0
}
```

PS C:\Users\Hayden Duong\Desktop\SIT314_729 - Software Architecture and Scalability for IOT\Module 3 - Enterprise Architectures\Technical Task> █

Picture 3 – It took 782 ms ~ 0.782 second to store the sensor data to database.



```
PS C:\Users\Hayden Duong\Desktop\SIT314_729 - Software Architecture and Scalability for IOT\Module 3 - Enterprise Architectures\Technical Task> node iotsensor.js
```

```
{
  "id": 0, "name": "temperature_sensor", "address": "221 Burwood Hwy, Burwood, VIC 3125", "time": 1754380436041, "temperature": 18
}
```

218 ms

```
{
  id: 0,
  name: 'temperature_sensor',
  address: '221 Burwood Hwy, Burwood, VIC 3125',
  time: 2025-08-05T07:53:56.041Z,
  temperature: 18,
  _id: new ObjectId('6891b8947a85f0a65fbce2fc'),
  __v: 0
}
```

365 ms

```
{
  id: 0,
  name: 'temperature_sensor',
  address: '221 Burwood Hwy, Burwood, VIC 3125',
  time: 2025-08-05T07:53:57.043Z,
  temperature: 19,
  _id: new ObjectId('6891b8957a85f0a65fbce2fe'),
  __v: 0
}
```

```
{
  "id": 0, "name": "temperature_sensor", "address": "221 Burwood Hwy, Burwood, VIC 3125", "time": 1754380438055, "temperature": 27
}
```

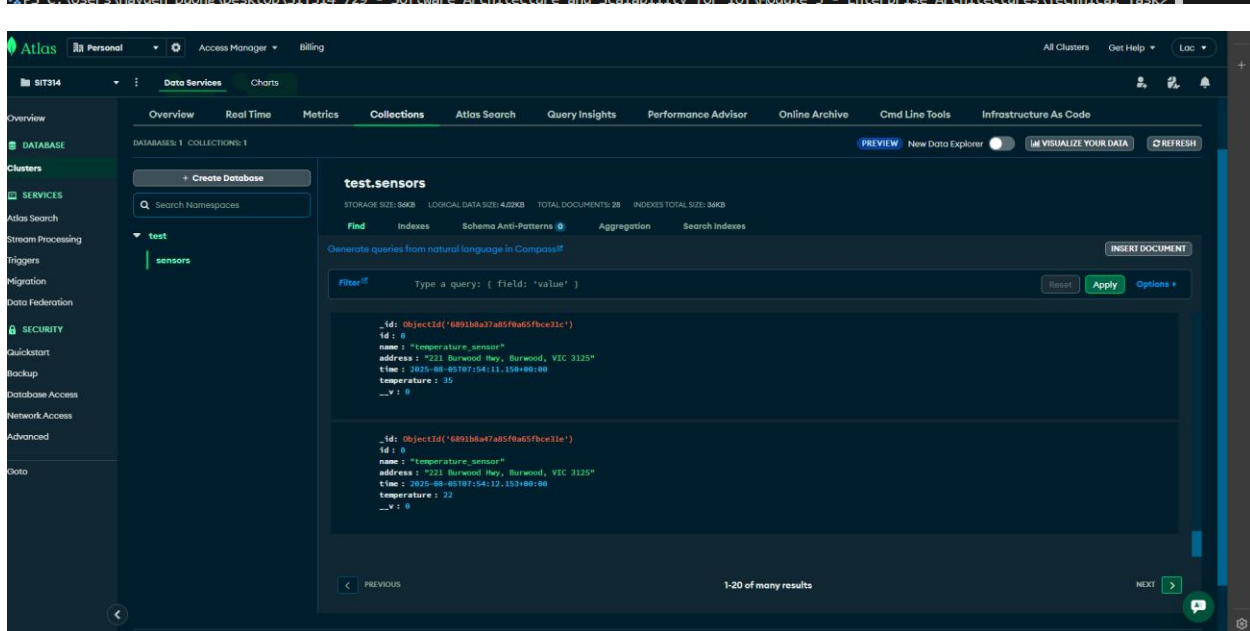
23 ms

```
{
  id: 0,
  name: 'temperature_sensor',
  address: '221 Burwood Hwy, Burwood, VIC 3125',
  time: 2025-08-05T07:53:58.055Z,
  temperature: 27,
  _id: new ObjectId('6891b8967a85f0a65fbce2ff'),
  __v: 0
}
```

```

{
  id: 0,
  name: 'temperature_sensor',
  address: '221 Burwood Hwy, Burwood, VIC 3125',
  time: 2025-08-05T07:54:18.190Z,
  temperature: 17,
  _id: new ObjectId('6891b8aa7a85f0a65fbce32a'),
  __v: 0
}
{"id":0,"name":"temperature_sensor","address":"221 Burwood Hwy, Burwood, VIC 3125","time":1754380459192,"temperature":39}
24 ms
{
  id: 0,
  name: 'temperature_sensor',
  address: '221 Burwood Hwy, Burwood, VIC 3125',
  time: 2025-08-05T07:54:19.192Z,
  temperature: 39,
  _id: new ObjectId('6891b8ab7a85f0a65fbce32c'),
  __v: 0
}
{"id":0,"name":"temperature_sensor","address":"221 Burwood Hwy, Burwood, VIC 3125","time":1754380460203,"temperature":33}
24 ms
{
  id: 0,
  name: 'temperature_sensor',
  address: '221 Burwood Hwy, Burwood, VIC 3125',
  time: 2025-08-05T07:54:20.203Z,
  temperature: 33,
  _id: new ObjectId('6891b8ac7a85f0a65fbce32e'),
  __v: 0
}

```



Picture 4, 5, 6 – Generated sensor data are stored successfully on MongoDB.

Week 6

A – Class Discussions

1 - What is the importance of software architecture?

- It provides a blueprint / diagram for a software system.
- It defines the system's structure, components, their relationships, and the principles guiding its design and evolution.
- A well-defined architecture ensures the system is scalable, maintainable, and reliable.
 - It helps manage complexity, improves communication among team members, and facilitates decision-making by providing a clear framework.

2 - What is layered architecture?

- Or the n-tier architecture pattern, organizes a system into horizontal layers, with each layer performing a specific function.
- Each layer only communicates with the layer directly above it and the one directly below it, creating a structured, modular design.

3 - Why do we need new architectures for IoT?

- Traditional software architectures often aren't suitable for the Internet of Things (IoT) because IoT systems have unique requirements:
 - IoT involves a massive number of devices, generates vast amounts of data, and operates in diverse, often resource-constrained environments.
 - Traditional architectures can't handle the scale, heterogeneity, and real-time processing needs of IoT.
 - New architectures are needed to manage device connectivity, data ingestion from millions of sensors, and enable edge computing, where data is processed closer to the source to reduce latency and bandwidth usage.

4 - What are the key elements and advantages of the oneM2M IoT Standardized Architecture?

- Common Service Functions (CSFs):

- These are a set of shared services that provide functionalities like security, discovery, and data management.
- They prevent developers from having to build these functions from scratch.
- Application Entities (AEs):
 - These are the application logic for devices and services, such as a smart home application or a traffic monitoring system.
- Middle layer:
 - This layer acts as the backbone, connecting different IoT devices and applications.
- The main advantage of oneM2M is its interoperability which creates a unified framework that allows devices and applications from different vendors to communicate seamlessly.

5 - What are the key elements and advantages of The IoT World Forum (IoTWF) Standardized Architecture

- A seven-layer model designed to provide a comprehensive view of an IoT system. Its key elements are:
 - **Layer 1 - Physical Devices and Controllers Layer:**
 - Hardware.
 - **Layer 2 - Connectivity Layer:**
 - The communication network (e.g., Wi-Fi, cellular).
 - **Layer 3 - Edge Computing Layer:**
 - Pre-processing and analysis of data close to the source.
 - **Layer 4 - Data Accumulation Layer:**
 - The data storage and aggregation.
 - **Layer 5 - Data Abstraction Layer:**
 - The translation of data for higher-level applications.
 - **Layer 6 - Application Layer:**

- The actual applications (e.g., smart home apps, industrial control).

➤ **Layer 7 - Collaboration & Processes Layer:**

- The integration of IoT data with business processes.
- The primary advantage of this model is its holistic and structured approach.
- It provides a clear, logical breakdown of an IoT system, which helps in designing and deploying complex solutions.
 - It emphasizes the importance of edge computing for efficient data management.

6 - What are the key elements and advantages of Purdue Model for Control Hierarchy

- A reference architecture for Industrial Control Systems (ICS) and operational technology (OT) environments.
- Its key elements are a hierarchical structure that segregates IT from OT:
 - Level 0 - Physical Process:
 - The actual sensors and actuators.
 - Level 1 - Intelligent Devices:
 - Devices like PLCs that control the process.
 - Level 2 - Control Systems:
 - Supervises Level 1, e.g., SCADA systems.
 - Level 3 - Site Operations:
 - Manage production operations.
 - Level 4 - Business Logistics Systems:
 - Traditional IT systems, e.g., ERP.
- The main advantage of the Purdue Model is the enhanced security it provides by creating a clear boundary between the operational network (OT) and the enterprise network (IT).
 - This isolation helps protect critical industrial processes from cyberattacks originating from the business network.

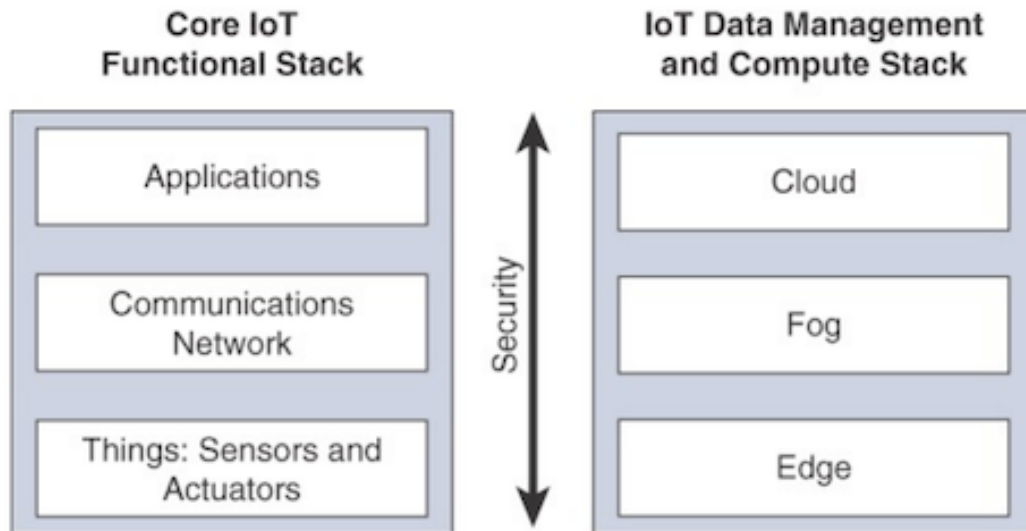
7 - What are the key elements and functionalities of:

- Layer 1 - Sensors and Actuators
 - Key Elements:

- Sensors (e.g., temperature, pressure, motion) that collect data from the physical world.
 - Actuators (e.g., motors, valves) that act upon the physical world based on instructions.
- Functionalities:
 - This is the physical layer of the IoT.
 - Its primary function is to sense environmental conditions and actuate changes.
 - Sensors convert physical phenomena into electrical signals, and actuators convert electrical signals back into physical actions.
- Layer 2 – Communications Network
 - Key Elements:
 - Gateways, routers, and network protocols (e.g., Wi-Fi, Bluetooth, Zigbee, cellular networks).
 - Functionalities:
 - This layer provides the connectivity that allows devices to transmit data to the cloud or other devices.
 - It handles the routing and transmission of data packets, ensuring reliable and secure communication between the physical devices and the application layer.
- Layer 3 - Applications and Analytics
 - Key Elements:
 - Data analytics platforms, cloud servers, and user-facing applications (e.g., mobile apps, dashboards).
 - Functionalities:
 - This layer is where the intelligence of the IoT system resides.
 - It processes, analyzes, and stores the data collected from the lower layers.
 - Applications provide the user interface for monitoring and controlling devices, while analytics engines derive insights from the data, enabling automation, predictive maintenance, and informed decision-making.

B – Group Activities

In this task, we will think about different applications in the context of the Simplified IoT Architecture discussed this week.



Tasks

For each of the following, what functionality and what technology needs to be at the i. Things Layer, ii. Communications layer, and iii. Applications layer.

1. A room temperature sensor for the room you are in.

Things **Layer:**

- **Functionality:** To measure the ambient temperature of the room.
- **Technology:** A temperature sensor, like a thermistor or a thermocouple, embedded within a device.

Communications **Layer:**

- **Functionality:** To transmit the temperature data to a network or application.
- **Technology:** Wireless protocols like Wi-Fi or Bluetooth Low Energy (BLE), or a low-power, wide-area network (LPWAN) protocol like LoRaWAN.

Applications Layer:

- **Functionality:** To display the current temperature, log historical data, or trigger an action, like turning on an air conditioner if the temperature exceeds a certain threshold.
- **Technology:** A mobile app, a web dashboard, or a cloud-based platform that processes and visualizes the data.

2. The status of a self-driving car.

Things Layer:

- **Functionality:** To collect real-time data on the car's speed, location, engine status, and surrounding environment.
- **Technology:** A combination of sensors, including GPS, radar, LIDAR, and cameras, as well as an onboard computer.

Communications Layer:

- **Functionality:** To transmit the collected data to a central monitoring system and receive instructions from it.
- **Technology:** High-speed cellular networks (4G/5G) for reliable and low-latency communication.

Applications Layer:

- **Functionality:** To monitor the car's status, analyze driving data, and provide over-the-air software updates.
- **Technology:** A fleet management system or a cloud-based application that processes and analyzes the data for performance and safety insights.

3. The location of a person.

Things Layer:

- **Functionality:** To determine the person's location.
- **Technology:** A GPS receiver in a smartphone, a wearable device, or a dedicated tracking device.

Communications Layer:

- Functionality: To transmit the location coordinates.
- Technology: Cellular networks (e.g., 4G, 5G) or satellite communication for widespread coverage.

Applications Layer:

- Functionality: To display the person's location on a map and provide location-based services.
- Technology: A mapping application like Google Maps or a dedicated tracking app.

4. A weather update for the location.

Things Layer:

- Functionality: To measure weather conditions such as temperature, humidity, and atmospheric pressure.
- Technology: Weather station sensors for temperature, humidity, pressure, and wind speed.

Communications Layer:

- Functionality: To transmit the weather data to a central server.
- Technology: Internet connectivity via Wi-Fi or Ethernet to upload data to a cloud service.

Applications Layer:

- Functionality: To provide a weather forecast and display current conditions.
- Technology: A weather application or a website that processes data from multiple stations to provide a comprehensive forecast.

5. The status of a smart vacuum cleaner.

Things Layer:

- Functionality: To monitor battery level, cleaning mode, and internal sensors for obstacle detection.

- Technology: An embedded processor, battery sensor, and various obstacle sensors (e.g., infrared or ultrasonic).

Communications Layer:

- Functionality: To communicate with a user's mobile device or a smart home hub.
- Technology: Wi-Fi or Bluetooth to connect to a local network.

Applications Layer:

- Functionality: To allow users to remotely control the vacuum cleaner, schedule cleanings, and view its status.
- Technology: A dedicated mobile app for the smart vacuum cleaner.

6. A TV program change request for a TV.

Things Layer:

- Functionality: To receive and execute a command to change the channel.
- Technology: An infrared (IR) receiver or a network interface card (NIC) within the TV.

Communications Layer:

- Functionality: To transmit the channel change command from a remote device to the TV.
- Technology: Wi-Fi or Bluetooth for smart TVs, or infrared signals for traditional remote controls.

Applications Layer:

- Functionality: To provide a user interface for selecting and sending channel change commands.
- Technology: A mobile application, a smart home hub, or a TV remote control.

7. The request message to set an air-conditioning setting.

Things Layer:

- **Functionality:** To receive a command to change a setting, like temperature or fan speed, and actuate the change.
- **Technology:** The air-conditioning unit with a built-in actuator and a network interface.

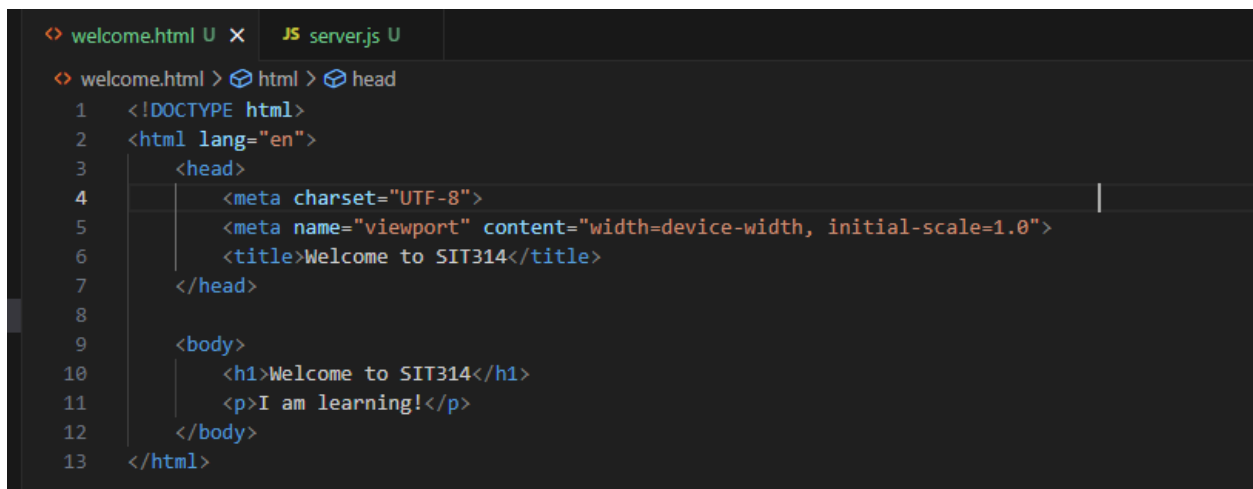
Communications Layer:

- **Functionality:** To transmit the command from a user's device to the air conditioner.
- **Technology:** Wi-Fi or Z-Wave for smart home integration.

Applications Layer:

- **Functionality:** To provide a user interface to set the desired temperature and other settings and send the command to the air conditioner.
- **Technology:** A smart thermostat mobile app or a smart home hub.

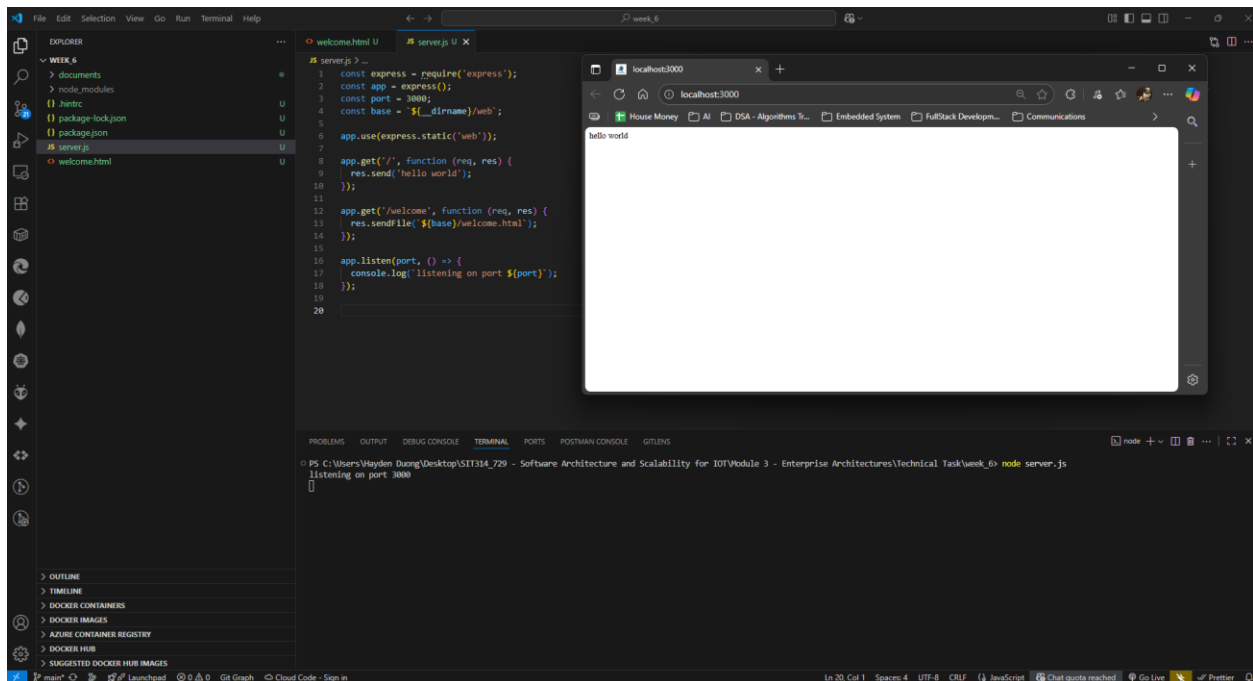
C - Technical Tasks

A screenshot of a code editor with a dark theme. The editor has two tabs at the top: 'welcome.html U' and 'JS server.js U'. The 'welcome.html' tab is active, showing the following HTML code:

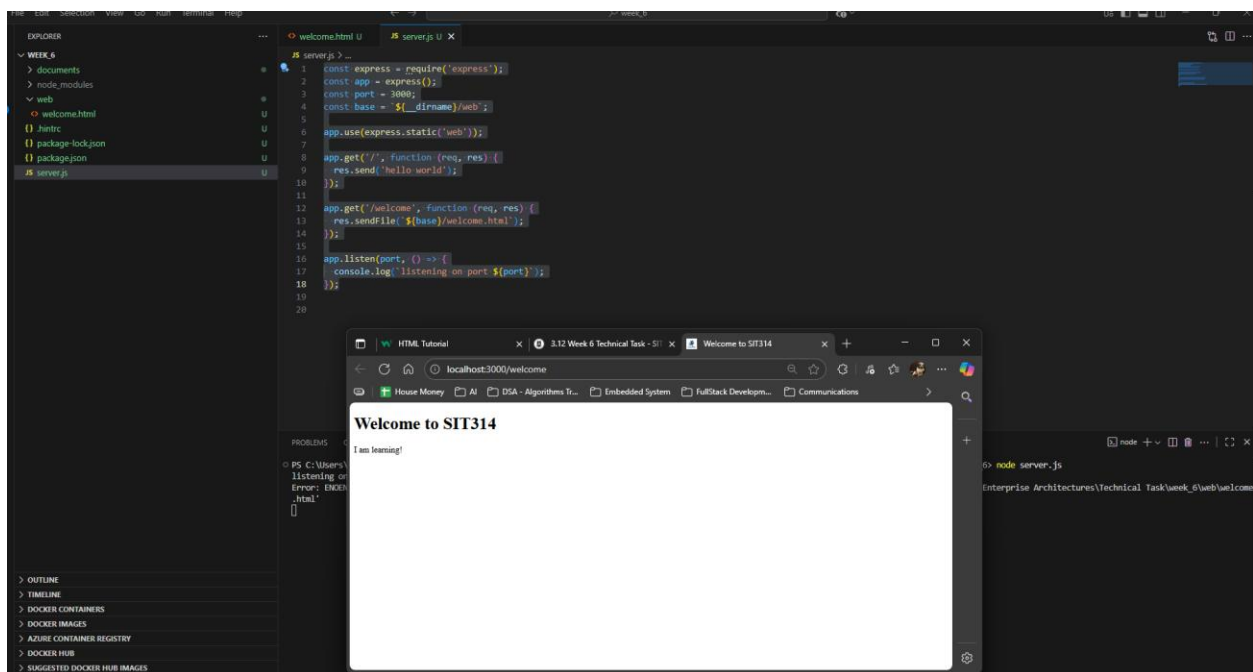
```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Welcome to SIT314</title>
  </head>
  <body>
    <h1>Welcome to SIT314</h1>
    <p>I am learning!</p>
  </body>
</html>
```

The code is numbered from 1 to 13 on the left side of the editor.

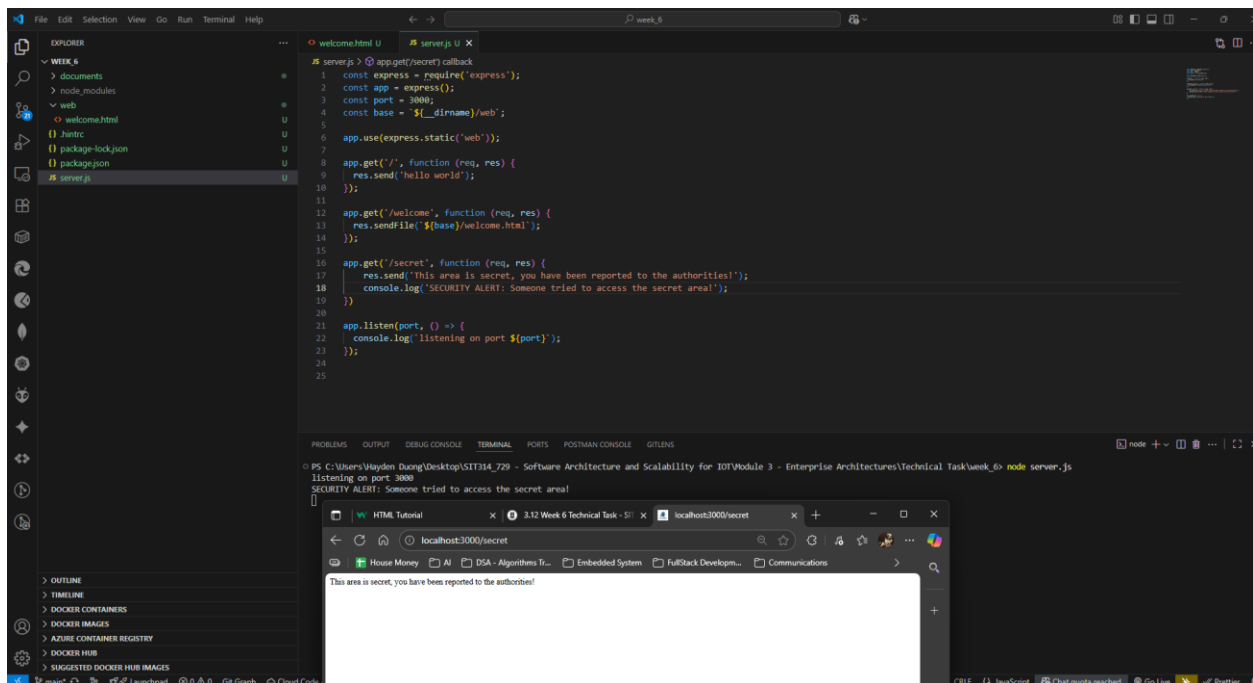
Picture 1 – Code of 'welcome.html'



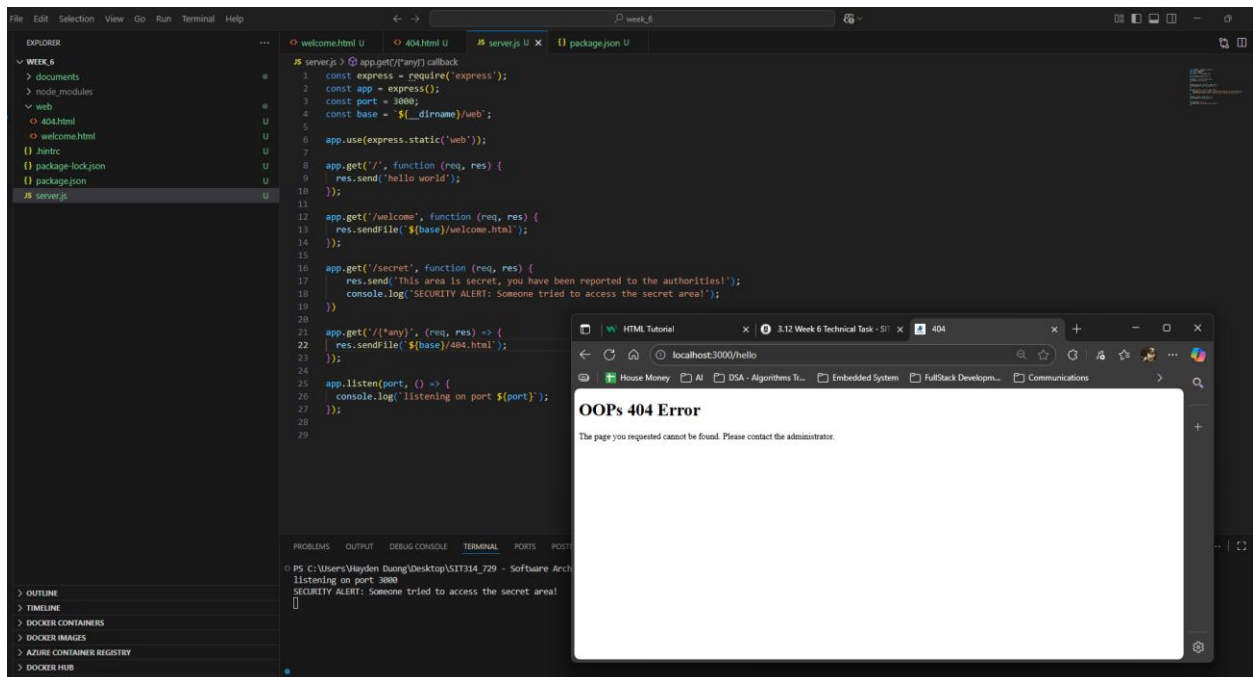
Picture 2 – Code of 'server.js' & result from running 'node server.js'



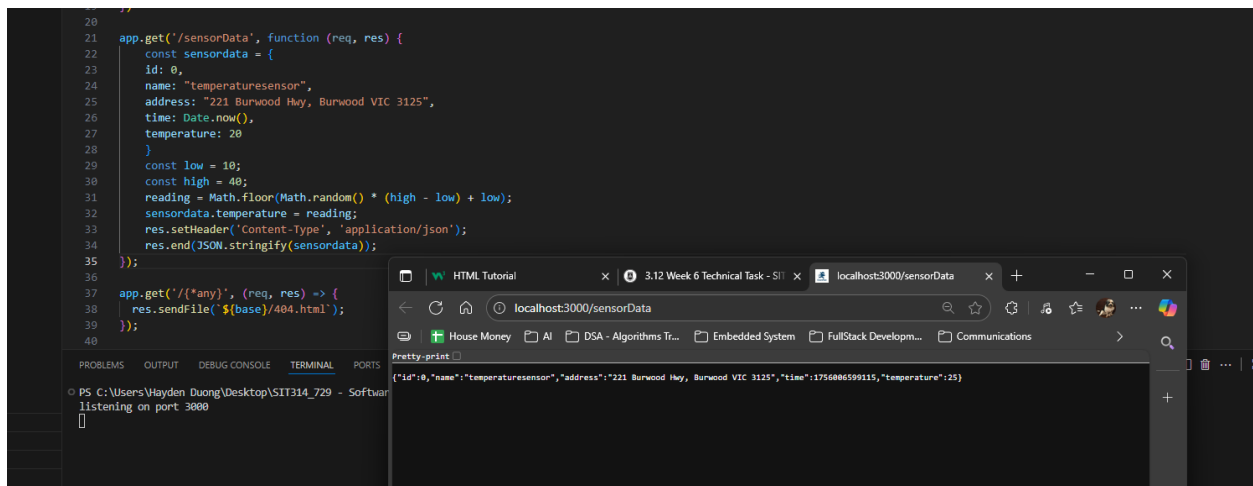
Picture 3 – Successfully displayed the '/welcome'



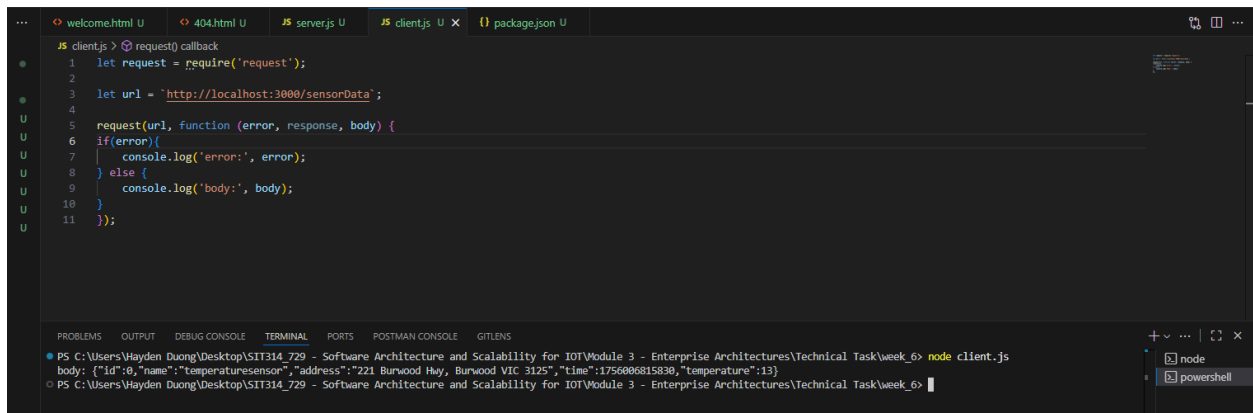
Picture 4 - Successfully displayed the result from '/secret'



Picture 5 - Successfully displayed the result from '/{*any}'



Picture 6 - Successfully displayed the result from `/sensorData`



Picture 7 - Successfully displayed the result from `'node client.js'`

The screenshot shows a VS Code editor with a file explorer on the left containing 'welcome.html', '404.html', 'server.js', 'client.js', and 'package.json'. The 'client.js' file is open in the editor, showing a JavaScript function that makes an HTTP request and logs the response. The terminal at the bottom shows the command 'node client.js' being executed, and the output displays the JSON response from the server, including sensor ID, name, address, time, and temperature.

```
4 request(url, function (error, response, body) {
5   if(error){
6     console.log('error:', error);
7   } else {
8     let sensorData = JSON.parse(body);
9     console.log(sensorData);
10    console.log('Sensor ID: ${sensorData.id}');
11    console.log('Sensor Name: ${sensorData.name}');
12    console.log('Sensor Address: ${sensorData.address}');
13    console.log('Timestamp: ${new Date(sensorData.time).toLocaleString()}');
14    console.log('Temperature: ${sensorData.temperature}°C');
15  }
16 });
```

```
PS C:\Users\Hayden Duong\Desktop\SIIT314_729 - Software Architecture and Scalability for IOT\Module 3 - Enterprise Architectures\Technical Task\week_6> node client.js
body: {"id":0,"name":"Temperaturesensor","address":"221 Burwood Hwy, Burwood VIC 3125","time":"175606915838","temperature":13}
{
  id: 0,
  name: 'temperaturesensor',
  address: '221 Burwood Hwy, Burwood VIC 3125',
  time: 175606917690,
  temperature: 33
}
Sensor ID: 0
Sensor Name: temperaturesensor
Sensor Address: 221 Burwood Hwy, Burwood VIC 3125
Timestamp: 175606917690
Temperature: 33°C
PS C:\Users\Hayden Duong\Desktop\SIIT314_729 - Software Architecture and Scalability for IOT\Module 3 - Enterprise Architectures\Technical Task\week_6> node client.js
{
  id: 0,
  name: 'temperaturesensor',
  address: '221 Burwood Hwy, Burwood VIC 3125',
  time: 175606937741,
  temperature: 32
}
Sensor ID: 0
Sensor Name: temperaturesensor
Sensor Address: 221 Burwood Hwy, Burwood VIC 3125
Timestamp: 8/24/2025, 1:42:17 PM
Temperature: 32°C
PS C:\Users\Hayden Duong\Desktop\SIIT314_729 - Software Architecture and Scalability for IOT\Module 3 - Enterprise Architectures\Technical Task\week_6>
```

Picture 8 - Successfully displayed the result from modified 'client.js'

The screenshot shows a VS Code editor with a file explorer on the left containing 'welcome.html', '404.html', 'server.js', 'client.js', and 'package.json'. The 'server.js' file is open in the editor, showing a JavaScript function that sets up an Express server and listens for requests. The terminal at the bottom shows the command 'node server.js' being executed, and the output displays the server listening on port 3001 and the results of the 'lightOn' and 'lightOff' requests.

```
1 const express = require('express');
2 const app = express();
3
4 const port = 3001;
5
6 app.get('/lightOn', function (req, res) {
7   console.log("Switching Light On");
8 });
9
10 app.get('/lightOff', function (req, res) {
11   console.log("Switching Light Off");
12 });
13
14 app.listen(port, () => {
15   console.log(`listening on port ${port}`);
16 });
```

```
PS C:\Users\Hayden Duong\Desktop\SIIT314_729 - Software Architecture and Scalability for IOT\Module 3 - Enterprise Architectures\Technical Task\week_6> node server.js
listening on port 3001
Switching Light On
Switching Light Off
```

Picture 9 - Successfully displayed the results from task 6c

Module Summary

From this module, I've learned that software architecture is the fundamental blueprint for a software system, defining its structure, components, and the principles guiding its design and evolution.

Well-defined architecture is crucial because it ensures the system is scalable, maintainable, and reliable.

I've also learned that traditional architectures aren't suitable for IoT systems due to their unique requirements, such as handling a massive number of devices and processing vast amounts of data in diverse environments. To address these challenges, I've studied several key IoT architectural models:

- **Layered Architecture:** This model organizes a system into horizontal layers, with each layer communicating only with the layers directly above and below it. This approach creates a structured, modular design.
- **Standardized IoT Architectures:** I've gained an understanding of the key elements and advantages of the **oneM2M IoT Standardized Architecture** and the **IoT World Forum (IoTWF) Standardized Architecture**. Both models aim to provide interoperability and a clear, logical breakdown of IoT systems.
- **Purdue Model:** I've learned about this reference architecture for Industrial Control Systems (ICS) and its hierarchical structure, which segregates IT and operational technology (OT). This separation enhances security by protecting critical industrial processes from cyberattacks.

Furthermore, I have explored key concepts related to data in IoT:

- **Big Data:** I understand the "Six V's" of Big Data—Volume, Velocity, Variety, Veracity, Value, and Variability—and how it is generated by humans, machines, and IoT devices.
- **Cloud Computing and NoSQL:** I've learned that IoT devices generate massive amounts of data, and **Cloud Computing** provides the storage and processing power needed to handle it.

NoSQL databases are a perfect fit for scalable IoT applications because they can handle unstructured, distributed, and high-velocity data efficiently.

- **Data Warehousing:** I've learned that data warehousing is the process of collecting and storing structured data in a centralized repository optimized for analytics and

reporting. It helps address the challenges of massive IoT data volume and multiple data sources, providing a way to store long-term data for trend detection and historical analysis.

These are equally important as they are the foundation for the upcoming modules and also help me start preparing for task D.